

Model-Theoretic Quantifier Elimination

A Lean formalization over `mathlib4`

Yağız Kaan Aydoğdu, Yusuf Demir, Salih Erdem Koçak

Advisors: Ayhan Günaydın, Metin Ersin Arıcan

Contents

1. Model Theory
2. Type Theory
3. Formalization in Lean

Model Theory

What is Model Theory?

- Model theory is a branch of mathematical logic where we study mathematical structures by considering the first-order sentences true in those structures and the sets definable by first-order formulas.
- Given concrete mathematical structures, we can obtain new information about the structure using model-theoretic techniques.

Language

- In mathematical logic, we use first-order languages to describe mathematical structures.
- A *language* $\mathcal{L}$ consists of:
 - a set of function symbols $\mathcal{F}$
 - a set of relation symbols $\mathcal{R}$
 - a set of constant symbols $\mathcal{C}$
- Examples:
 - The language of rings $\mathcal{L}_r = \{+, -, \cdot, 0, 1\}$
 - The language of orders $\mathcal{L}_o = \{<\}$

Structure

- A structure interprets the symbols of a language on a domain.
- Formally, an $\mathcal{L}$ -structure $\mathcal{M}$ consists of:
 - a non-empty set M (the domain/universe),
 - for each function symbol $f \in \mathcal{F}$, a function $f^{\mathcal{M}} : \mathcal{M}^n \rightarrow \mathcal{M}$,
 - for each relation symbol $R \in \mathcal{R}$, a relation $R^{\mathcal{M}} \subseteq \mathcal{M}^n$,
 - for each constant symbol $c \in \mathcal{C}$, an element $c^{\mathcal{M}} \in \mathcal{M}$.
- Examples: $(\mathbb{N}, 0, 1, +, \cdot)$, $(\mathbb{Q}, \leq)$.

Terms

- An $\mathcal{L}$ -term is:
 - a constant symbol c ,
 - a variable v_i
 - or a function symbol f applied to terms $t_1, \dots, t_n$.

Formula and Sentence

- An *atomic $\mathcal{L}$ -formula* is a relation symbol applied to terms, or an equality between terms.
- *$\mathcal{L}$ -formulas* are built from atomic formulas using logical connectives and quantifiers.
- An *$\mathcal{L}$ -sentence* is a formula with no free variables.
- Example: $\exists z (x < z \wedge z < y)$ is formula but not a sentence, as it has free variables x and y .

Satisfaction

- We say that an $\mathcal{M}$ *satisfies* a formula φ , written $\mathcal{M} \models \varphi$, if φ holds in $\mathcal{M}$.
- Formally, the satisfaction relation is defined inductively on the structure of formulas.
- Example: $(\mathbb{Q}, <) \models \forall x \forall y (x < y \implies \exists z x < z \wedge z < y)$.

Theory and Model

- An $\mathcal{L}$ -theory is a set of $\mathcal{L}$ -sentences.
- We say that a structure $\mathcal{M}$ is a *model* of a theory T , written $\mathcal{M} \models T$, if $\mathcal{M}$ satisfies every sentence in T .
- A theory is *satisfiable* if it has a model.
- Moreover, we say that a theory T satisfies a sentence φ , written $T \models \varphi$, if every model of T satisfies φ .

Quantifier Elimination

- A theory T has *quantifier elimination* if for every formula φ , there is a quantifier-free formula ψ such that

$$T \models \varphi \leftrightarrow \psi$$

Dense Linear Orders without Endpoints

- Let $\mathcal{L}_o = \{<\}$ be the language of orders.
- Let DLO be the $\mathcal{L}_o$ -theory consisting of the following sentences:

$\forall x \neg(x < x)$	(Irreflexivity)
$\forall x \forall y \forall z (x < y \wedge y < z \rightarrow x < z)$	(Transitivity)
$\forall x \forall y (x < y \vee x = y \vee y < x)$	(Linearity)
$\forall x \forall y (x < y \rightarrow \exists z (x < z \wedge z < y))$	(Density)
$\forall x \exists y \exists z (y < x \wedge x < z)$	(No endpoints)

- Examples of models of DLO: $(\mathbb{Q}, <)$, $(\mathbb{R}, <)$, $((0, 1), <)$ with the usual order.

Does DLO have Quantifier Elimination?

- DLO has quantifier elimination.
- Every formula in the language $\{<\}$ is equivalent over DLO to a quantifier-free formula.
- Examples:

$$\exists z (x < z \wedge z < y) \iff x < y$$

$$\exists z (x < z \wedge w < z \wedge z < y) \iff x < y \wedge w < y$$

$$\exists z (x < z \wedge z < y \wedge \neg(z < w)) \iff x < y \wedge (w < y \vee w = y)$$

Type Theory

Type Theory

- We work in the sense of Martin-Löf Type Theory: a constructive foundation of mathematics.
- The basic syntactic form is a *judgement*, not a proposition.
- Every object and expression has a (unique) type, statically determined.
- This is the foundation used by proof assistants such as Lean, Agda, and Coq.

proofs as programs propositions as types

Type Theory vs Set Theory

Set theory

- $3 \in \mathbb{N}$ is a *proposition*.
- Membership is the basic relation.
- Functions are derived: subsets of $A \times B$ with certain properties.
- One may ask encoding-dependent questions, e.g. $\mathbb{N} \cap \text{Bool} = \emptyset$?

Type theory

- $3 : \mathbb{N}$ is a *judgement* (static information).
- Typing is the basic judgement; $a : A$ is not a proposition.
- Functions are primitive: given $A, B : \text{Type}$, we form $A \rightarrow B : \text{Type}$.
- Type theory is extensional in the sense that we cannot talk about implementation details.

Judgements

- A *judgement* is static information that the type system can check.
- The central judgement is $a : A$: “ a is an element of type A ”.
- A *context* Γ records the assumptions currently in scope.

$$\Gamma = x : A, f : A \rightarrow B \vdash f x : B$$

Functions

- Given $A, B : \text{Type}$, the function type $A \rightarrow B : \text{Type}$ is primitive.
- A function is defined by its action on inputs, e.g. $f : \mathbb{N} \rightarrow \mathbb{N}$ with $f\ x \equiv x + 3$.
- Anonymous functions use λ -abstraction: $\lambda x. x + 3 : \mathbb{N} \rightarrow \mathbb{N}$.

$$(\lambda x. x + 3)\ 2 \equiv 2 + 3 \quad (\beta\text{-reduction})$$

- Every function has exactly one argument; multiple arguments use currying:
 $g : \mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$, so $g\ 3\ 2 : \mathbb{N}$.
- Application associates to the left; $\rightarrow$ associates to the right.

Products and Sums

- Given $A, B : \text{Type}$, we form the product $A \times B$ and the sum $A + B$.
- $(a, b) : A \times B$ if $a : A$ and $b : B$.
- $\text{left } a : A + B$ if $a : A$; $\text{right } b : A + B$ if $b : B$.
- The unit type 1 has a single element $()$; the empty type 0 has no elements.

Propositions as Types

- A proposition P is represented as a type $P : \text{Type}$.
- A proof of P is a term $p : P$, evidence for the proposition.

to prove P is to construct $p : P$

Curry–Howard Translation

$$\llbracket P \Rightarrow Q \rrbracket \equiv \llbracket P \rrbracket \rightarrow \llbracket Q \rrbracket$$

$$\llbracket P \wedge Q \rrbracket \equiv \llbracket P \rrbracket \times \llbracket Q \rrbracket$$

$$\llbracket \text{True} \rrbracket \equiv 1$$

$$\llbracket P \vee Q \rrbracket \equiv \llbracket P \rrbracket + \llbracket Q \rrbracket$$

$$\llbracket \text{False} \rrbracket \equiv 0$$

$$\llbracket \forall x : A. P(x) \rrbracket \equiv \prod x : A. \llbracket P(x) \rrbracket$$

$$\llbracket \exists x : A. P(x) \rrbracket \equiv \sum x : A. \llbracket P(x) \rrbracket$$

Dependent Types

- A *dependent type* is a family $B : A \rightarrow \text{Type}$ indexed by A .
- Π -types generalise function types: the codomain may depend on the domain.
- Σ -types generalise products: the second component may depend on the first.

$$\Pi x : A. B(x) \qquad \Sigma x : A. B(x)$$

- Example: $\text{zeroes} : \Pi n : \mathbb{N}. A^n$ — assigns to each length n a tuple of n elements of type A .
- Example: $\text{List } A \equiv \Sigma n : \mathbb{N}. A^n$.
- Predicates are dependent types: e.g. $\text{Prime} : \mathbb{N} \rightarrow \text{Type}$.

Typed Predicate Logic

- We use a *typed* predicate logic: quantifiers range over a type A .
- Other connectives are defined: $\neg P \equiv P \rightarrow 0$,
 $P \Leftrightarrow Q \equiv (P \rightarrow Q) \times (Q \rightarrow P)$.
- Not all classical tautologies hold, e.g. $\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$.
- The law of excluded middle $P \vee \neg P$ is not provable in general.

$\exists x : A. P(x) \rightsquigarrow \Sigma x : A. P(x) =$ a pair (a, p) with $a : A$, $p : P(a)$.

- A proof of $\exists x : A. P(x)$ provides an explicit witness $a : A$ and evidence $p : P(a)$!

Example: Distributivity of Conjunction over Disjunction

Goal: $P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$.

For $P, Q, R : \text{Type}$, define maps

$$f : P \times (Q + R) \rightarrow (P \times Q) + (P \times R),$$

$$g : (P \times Q) + (P \times R) \rightarrow P \times (Q + R)$$

by pattern matching on constructors:

$$f(p, \text{left } q) \equiv \text{left } (p, q),$$

$$f(p, \text{right } r) \equiv \text{right } (p, r)$$

$$g(\text{left } (p, q)) \equiv (p, \text{left } q),$$

$$g(\text{right } (p, r)) \equiv (p, \text{right } r)$$

The pair (f, g) is evidence for the logical equivalence!

The Equality Type

- Given $a, b : A$, the equality type $a =_A b : \text{Type}$ is generated by one constructor:

$$\text{refl} : \prod x : A. x =_A x$$

- A proof $p : a =_A b$ is evidence that a and b are equal.
- From refl we derive symmetry, transitivity, and congruence:

$$\text{sym} : \prod x, y : A. x =_A y \rightarrow y =_A x$$

$$\text{trans} : \prod x, y, z : A. x =_A y \rightarrow y =_A z \rightarrow x =_A z$$

Dimensions of Types

- The type A with its equality types forms a *groupoid*: `refl` is identity, `sym` gives inverses, `trans` composes paths.
- Types are classified by *dimension* (truncation level): how much higher equality structure they carry.

Dimension	Name
−2	contractible types
−1	propositions
0	sets
1	groupoids

- A *proposition* has at most one element; a *set* has propositional equalities; a *groupoid* has set-like equalities between paths.

Extensionality and Univalence

- Example: $1 + 2$ and $2 + 1$ are indistinguishable, yet not definitionally equal.

A map $f : A \rightarrow B$ is an *equivalence* if it has a two-sided inverse up to equality. Write

$$A \simeq B : \equiv \Sigma f : A \rightarrow B. \text{isEquiv } f.$$

Path induction gives $\text{eq2equiv} : A =_{\text{Type}} B \rightarrow A \simeq B$.

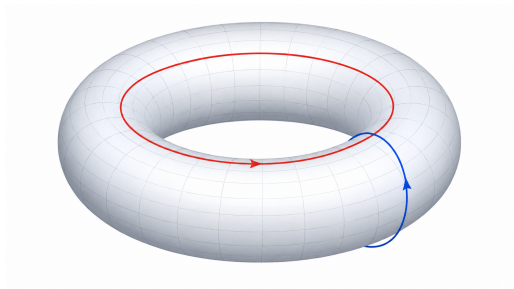
Univalence axiom: eq2equiv is itself an equivalence:

$$\text{isEquiv}(\text{eq2equiv}) \quad \Longleftrightarrow \quad (A \simeq B) \simeq (A =_{\text{Type}} B)$$

Consequence: $A \simeq B \rightarrow A =_{\text{Type}} B$ — equivalent types may be treated as equal.

Towards Homotopy Type Theory

- Types carry higher-dimensional structure: paths, paths between paths, ...
- This ω -groupoid view connects logic, computation, and topology.
- Lean's core is based on intensional type theory; univalence and HoTT are active research areas.



Formalization in Lean

```

/-- A formula is equivalent to a quantifier-free formula over a theory. -/
def IsQFEquivalent (T : L.Theory) {α : Type*} (φ : L.Formula α) : Prop :=
  ∃ ψ : L.Formula α, ψ.IsQF ∧ φ ⇔[T] ψ

/-- A theory has quantifier elimination if every formula is equivalent,
over the theory, to a quantifier-free formula. -/
def HasQuantifierElimination (T : L.Theory) : Prop :=
  ∀ {α : Type} (φ : L.Formula α), T.IsQFEquivalent φ

```

```
-- The theory of dense linear orders without endpoints
has quantifier elimination. -/
theorem dlo_hasQuantifierElimination :
  Language.order.dlo.HasQuantifierElimination := by
  ...
```

Thank you for listening!



Scan the QR code to checkout our PR in `mathlib4` GitHub repository!